

ClipHub

Universal Clipboard & Secure File Sharing Platform Product Requirements Document

Version 1.0

Prepared by: Yug Patel

Status: Final for Engineering Handoff

August 2026

Table of Contents

Table of Contents.....	2
1. Executive Summary.....	4
1.1 Product Vision.....	4
1.2 The "Why Now?" Context.....	4
1.3 Strategic Objectives.....	4
2. Problem Space & Pain Points	4
2.1 The Friction of Micro-Transfers.....	4
2.2 Security and Privacy Vulnerabilities.....	5
2.3 Cross-Ecosystem Barriers.....	5
3. Target Personas.....	5
Persona: The Multi-Device Developer - Software Engineer.....	5
Persona: The Cross-Platform Student - University Student.....	5
Persona: The Remote Freelancer - Designer / Consultant	5
4. Core Product Principles.....	6
5. Competitive Analysis	6
6. Core Features & Prioritization	7
6.1 MVP Scope (Version 1.0)	7
6.2 Deferred to V2 (Fast Follows).....	7
7. Detailed Feature Specifications.....	7
7.1 Feature: Ephemeral Links (Burn After Read).....	8
7.2 Feature: Password Protection & Client-Side Encryption	8
7.3 Feature: QR Code Local Hand-off	8
8. User Journeys & Flows.....	9
8.1 Flow 1: Anonymous Text Transfer (Desktop to Mobile)	9
8.2 Flow 2: Authenticated File Sharing with Revocation.....	10
9. Technical & Security Considerations.....	10
10. Success Metrics & KPIs	11
11. Product Roadmap	11
12. Risks and Mitigations.....	12

12.1 Phishing and Abuse.....	12
12.2 Infrastructure Cost Scaling.....	12

1. Executive Summary

1.1 Product Vision

ClipHub aims to eliminate the friction of moving text snippets and files between devices. In an increasingly fragmented hardware ecosystem, users routinely face absurd micro-frictions when trying to move a simple paragraph of text, a URL, or a temporary file from their laptop to their mobile device, or from a Windows workstation to an iPad. Instead of relying on clumsy workarounds like emailing oneself, utilizing self-messaging in Slack or WhatsApp, searching for a USB drive, or polluting permanent cloud storage services, ClipHub enables users to securely share clipboard content and files using ephemeral, temporary links, password protection, expiration controls, QR codes, and seamless cloud synchronization.

1.2 The "Why Now?" Context

The modern knowledge worker operates in a multi-device, multi-OS reality. However, device ecosystems have built walled gardens. Apple's Universal Clipboard works flawlessly—but strictly within the Apple ecosystem. For users bridging Windows, Android, macOS, and Linux, the clipboard remains isolated to the local machine. Current alternatives are either permanent (Google Drive/Dropbox, which require file management and cleanup) or insecure (Pastebin, which lacks robust privacy by default). There is a significant market gap for a high-speed, secure, ephemeral bridge between discrete computing environments.

1.3 Strategic Objectives

- Reduce 'Time-to-Transfer' (TTT) for small payloads (text/files) from an industry average of 45 seconds down to under 5 seconds.
- Establish a Zero-Trust transfer protocol where sensitive data can be moved across unowned devices safely via ephemeral, password-protected links.
- Build a frictionless acquisition loop through product-led growth (PLG) where every shared link acts as an acquisition mechanism.

2. Problem Space & Pain Points

2.1 The Friction of Micro-Transfers

When a user needs to move an authentication token, a block of code, a long URL, or a quick screenshot from their phone to their PC, the cognitive load is disproportionately high compared to the task's value. Users resort to 'hacks': sending Slack messages to themselves,

drafting emails they never send, or dropping files into heavy cloud drives. These actions pollute workspaces, consume unnecessary storage, and disrupt flow state.

2.2 Security and Privacy Vulnerabilities

Because official channels are high-friction, users turn to shadow IT. Pasting API keys or sensitive customer data into public pastebins, or sending them through personal WhatsApp channels, poses severe data leakage risks. Users lack a tool that is as fast as Pastebin but adheres to enterprise-grade security (encryption, self-destruction, access logging).

2.3 Cross-Ecosystem Barriers

Ecosystem lock-in is actively hostile to user productivity. While a developer may use a MacBook for coding, they might test on an Android device or collaborate with a team member on a Windows machine. Moving state across these boundaries without installing heavy MDM or synchronization clients is currently an unsolved problem in the consumer and prosumer space.

3. Target Personas

Persona: The Multi-Device Developer - Software Engineer

Primary Pain Point: Needs to move API keys, shell commands, and log outputs between their secure work laptop and their personal testing devices without committing to GitHub or leaving traces on corporate Slack.

Primary Use Case: Securely passing text snippets across OS boundaries using time-expiring links.

Why ClipHub wins them over: ClipHub respects ephemerality. An API key shared via ClipHub can be set to 'Burn after reading' or expire in 5 minutes, mitigating credential leakage.

Persona: The Cross-Platform Student - University Student

Primary Pain Point: Often studies in the library using public computers, but takes notes on a personal iPad or Android phone.

Primary Use Case: Scanning a QR code on the library computer screen to instantly grab a PDF or a copied block of research text onto their mobile device.

Why ClipHub wins them over: The QR code hand-off requires zero login on the public machine, ensuring their account is never compromised.

Persona: The Remote Freelancer - Designer / Consultant

Primary Pain Point: Needs to send quick design assets or temporary passwords to clients without forcing the client to sign up for a service or download a heavy application.

Primary Use Case: Generating password-protected links with an expiration date to send via email.

Why ClipHub wins them over: Professionalism and security. The client gets a clean, branded landing page to download the asset, and the freelancer gets peace of mind that the link will expire.

4. Core Product Principles

Every feature decision must be weighed against these core product tenets:

1. Speed is a Feature: The time from 'Intent to Share' to 'Link Generated' must be sub-second. Every additional click reduces user retention.
2. Ephemeral by Default: We are not a cloud storage company. We are a transport layer. Data should die naturally. Defaults must heavily favor short expiration times.
3. Zero-Knowledge Architecture (Where Applicable): If a user adds a password, we cannot read the payload. The password encrypts the data client-side.
4. Frictionless Consumption: The recipient of a ClipHub link should never be forced to create an account to view or download a payload.

5. Competitive Analysis

To position ClipHub effectively, we evaluate the current landscape across two axes: Setup Friction and Cross-Platform Capability.

Competitor	Cross-Platform	Security/Privacy	Setup Friction	ClipHub Advantage
Apple Universal Clipboard	No (Apple only)	High	Low	ClipHub is OS-agnostic.
Slack / WhatsApp (Self-DM)	Yes	Low (Permanent log)	Medium	ClipHub is ephemeral and does not pollute message history.
Pastebin	Yes	Low (Public by default)	Low	ClipHub offers password protection, file support, and an ad-free pro UI.
WeTransfer	Yes	Medium	High (Email)	ClipHub is

			required)	instant, supports text snippets, and uses QR codes for local hand-off.
--	--	--	-----------	--

6. Core Features & Prioritization

To validate the market quickly, we are strictly scoping the MVP to features that solve the immediate cross-device text and small-file transfer problem. Advanced team and enterprise features are deferred.

6.1 MVP Scope (Version 1.0)

- Anonymous Snippet Sharing: Paste text, get a link instantly.
- Temporary File Sharing: Upload files (up to 100MB) to generate a shareable link.
- Expiration Controls: Time-to-Live (TTL) settings (e.g., 1 hour, 24 hours) or 'Burn after read'.
- Password Protection: Symmetrical encryption utilizing a user-provided passphrase.
- QR Code Hand-off: Dynamically generated QR codes on the web app for instant mobile ingestion.
- Authenticated Dashboard: Basic user accounts to track active links and manually revoke access.

6.2 Deferred to V2 (Fast Follows)

- Real-time Cloud Clipboard Sync (Requires native desktop applications).
- End-to-End Encryption (E2EE) by default for all authenticated users.
- Team Workspaces (Shared clipboards).
- Browser Extensions for immediate contextual clipping.

7. Detailed Feature Specifications

This section details the expected behavior, business logic, and acceptance criteria for engineering handoff.

7.1 Feature: Ephemeral Links (Burn After Read)

Why: Security-conscious users need absolute certainty that sensitive data (like API keys or passwords) cannot be accessed by third parties after the intended recipient views it.

What: When creating a clip, the user can select an expiration trigger. If 'Burn After Read' is selected, the database record is hard-deleted the moment the payload is successfully served to a client via GET request.

Acceptance Criteria:

1. Given a user creates a 'Burn After Read' text clip, when recipient A opens the link, the text is displayed perfectly.
2. When recipient A refreshes the page, the server must return a 404/Expired state.
3. The payload must be wiped from the database; soft deletes (flagging as inactive) are unacceptable for this specific feature.

7.2 Feature: Password Protection & Client-Side Encryption

Why: Users need a way to send confidential files over public channels (like a Discord server or public Slack) where only the intended recipient, who has the out-of-band password, can decrypt it.

What: A toggle allowing the user to input a password. For maximum security, this password should ideally generate a cryptographic key client-side to encrypt the payload before it leaves the browser, ensuring ClipHub servers never hold the plaintext data.

Acceptance Criteria:

1. If a clip has a password, the landing page must prompt for the password before revealing any metadata (other than file size and type).
2. Incorrect password attempts must be rate-limited to prevent brute-force attacks (e.g., 5 attempts before a 15-minute lockout).

7.3 Feature: QR Code Local Hand-off

Why: Typing short-links from a computer screen into a mobile browser is high-friction.

What: Every generated clip link must have an adjacent 'Show QR' button. Scanning the QR code with a native mobile camera routes the user directly to the clip payload.

Acceptance Criteria:

1. The QR code must render locally in the DOM (e.g., via a React QR library) rather than relying on an external API, to ensure speed and privacy.
2. The QR modal must be prominent on desktop views, but hidden on mobile views (where scanning a screen with the same device is impossible).

8. User Journeys & Flows

8.1 Flow 1: Anonymous Text Transfer (Desktop to Mobile)

[User on Windows Desktop]

|

v

Navigates to ClipHub.com

|

v

Pastes text into main input area

|

v

Clicks "Create Clip" (Defaults applied: 24h expiration)

|

v

System generates unique short-link (e.g., clip.hb/x7F9a)

|

v

User clicks "Show QR Code"

|

v

[User opens iOS Camera] -> Scans QR Code

|

v

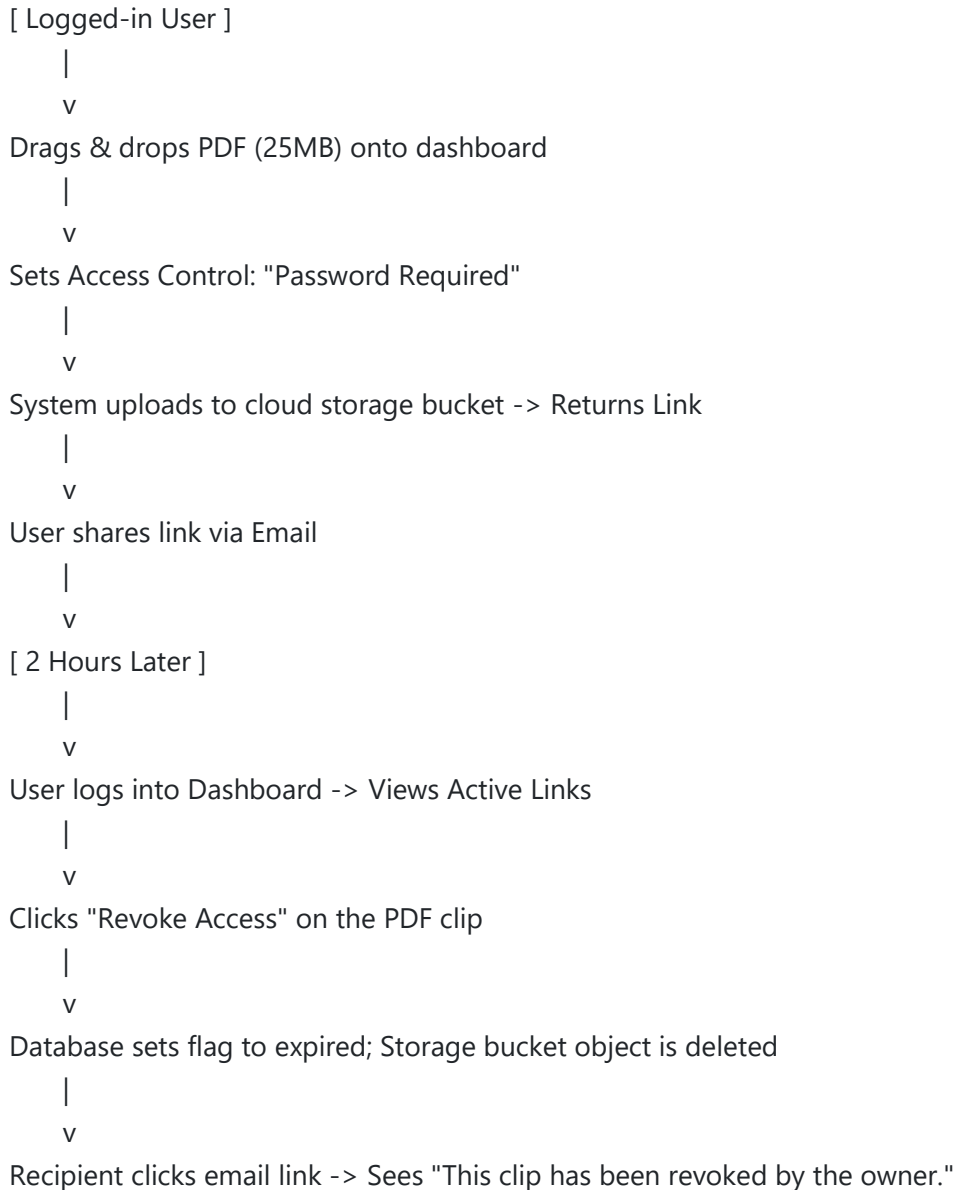
Mobile browser opens direct link

|

v

Payload is displayed with a "Copy to Clipboard" button

8.2 Flow 2: Authenticated File Sharing with Revocation



9. Technical & Security Considerations

While engineering will handle implementation details, the product requires the following non-functional constraints to ensure market viability:

Data Retention Policy (TTL): Storage costs are the primary risk factor for this business model. Unauthenticated files **MUST** have a hard maximum TTL of 24 hours. Authenticated users may have up to 7 days for MVP.

Malware & Abuse Prevention: Because we host user-uploaded files, the domain is at risk of being blacklisted by Google Safe Browsing if abused by bad actors. We must integrate a lightweight antivirus scanning API (e.g., ClamAV or VirusTotal integration) for all binary uploads before they become available for download.

Rate Limiting: To prevent infrastructure exhaustion, unauthenticated IP addresses must be strictly rate-limited (e.g., max 20 clips per hour).

10. Success Metrics & KPIs

To evaluate product-market fit post-launch, PM and Data teams will track the following metrics via Mixpanel/PostHog:

Metric Category	Key Performance Indicator (KPI)	Target (Launch + 30 Days)
Acquisition (PLG)	Virality Factor (Clips created by new users who discovered the app via a received link)	> 0.15
Activation	Time-to-First-Clip (Time from landing on site to generating a link)	< 10 seconds
Engagement	Clips per Active User per Week	Average > 4
System Health	Upload/Download Success Rate	99.9%

11. Product Roadmap

The strategic rollout for ClipHub is designed to capture individual users first, before moving upmarket to prosumers and teams.

Phase 1: Validation (MVP)	Phase 2: Retention (Growth)	Phase 3: Monetization (Pro)
• Anonymous text/file sharing • Ephemeral links & passwords • QR Code Hand-off • Basic User Dashboard	• Browser Extensions (Chrome/Firefox) • Native Mobile Apps (iOS/Android) • Cross-device push notifications	• End-to-End Encryption (E2EE) • Custom Domains for links • Team Workspaces & Audit Logs • API Access

12. Risks and Mitigations

12.1 Phishing and Abuse

Risk: Attackers use ClipHub links to bypass corporate email filters, masking phishing URLs or malware.

Mitigation: Implement automated heuristic scanning on text payloads containing URLs. Display a prominent 'Caution: This link is user-generated' interstitial warning for unauthenticated file downloads.

12.2 Infrastructure Cost Scaling

Risk: Generous file limits lead to rapid AWS S3 cost bloat without corresponding revenue.

Mitigation: Enforce aggressive automated TTL (Time-to-Live) sweeps via cron jobs to permanently delete expired payload blobs from S3 every hour. Implement hard caps on unauthenticated file sizes.

End of Product Requirements Document